CITYPULSE / INSTRUCTOR WALKTHROUGH

CityPulse: teach the connected delivery story

A browser-only script for showing one connected GitHub delivery environment without changing its baseline.

50	7	4	1
real issues	saved views	custom charts	named sprint

Lead a browser-only tour from a fictional city-service launch to its portfolio, sprint, dependencies, reporting, and review decisions. The GitHub environment is real; the citizen-service product and delivery story are fictional.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo

DO · CLICK PATH

1. Open the repository link in the normal authenticated browser session; confirm alvine-aurelio-org/citypulse-projects-demo.
2. Point to the Private badge and README overview, then locate OPEN THE SPRINT BOARD without editing anything.
3. Use page 2 to choose a route; keep this PDF beside the browser as the instructor script.

SHOW · FOCUS THE AUDIENCE

- 50 real issues, including 5 epics and 45 delivery children.
- 7 saved views, 4 custom charts, and 1 named sprint.
- One connected planning environment, not a deployed citizen portal.

EXPECTED · VERIFY ON SCREEN

- 50 issue cards; 40 open / 10 closed-completed.
- 45 native child links; 15 native dependency relationships.
- Scenario frozen at November 9, 2026 EOD.

SAY · INSTRUCTOR SCRIPT

"CityPulse is a fictional service launch. We will reuse the same real issues to explain scope, commitment, blockers, and risk. I will distinguish native GitHub evidence from illustrative history, and leave the environment ready for the next presenter."

ASK · THEN EXPLAIN

What should this tour prove?

That the planning environment connects decisions and evidence, not that a citizen-service application exists.

Protect the demo. Diagrams are schematics, not screenshots; the burndown reuses the repository SVG. Protect the native baseline.

Exit check. The correct private repository is open and the audience understands the fictional-product boundary.

Launch repository · Linked Project 11 · Sprint board

Choose the 35-minute or 15-minute route

Allow 30–40 minutes for the full script and audience questions. The table is a 15-minute fast track using printed page numbers; preparation and recovery are outside that running time.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11

DO · CLICK PATH

1. Prepare with page 3 before screen sharing. For the full tour, follow pages 4–20 in order.
2. For 15 minutes, follow the table; skip the optional move on page 21 and take questions after the close.
3. For a 5-minute executive alternative, use page 8 for 2 minutes, page 9 for 1, page 11 for 1, and page 24 for 1.
4. Keep the linked Sprint board, burndown report, and Insights tabs ready so navigation does not consume the discussion.

SHOW · FOCUS THE AUDIENCE

- Full route includes milestones, bug triage, native history, and revision-specific checks.
- Fast track prioritises the commitment, the gap, the blocker, and the decision.
- Page 22 supplies answers; page 23 supplies non-destructive recovery.

EXPECTED · VERIFY ON SCREEN

- Full tour: 30–40 minutes.
- Fast track: 15 minutes.
- Optional executive route: 5 minutes.

Printed pages	Fast-track focus	Minutes
Pages 4–6	Repository + portfolio	2
Pages 7–8	Delivery + Sprint 1	3
Page 9	Burndown	2
Pages 10–11	Blockers + At risk	2
Pages 12 and 14	Roadmap + workload	2
Pages 16 and 18–20	Insights + PRs + workflows	3
Page 24	Wrap and reset	1

SAY · INSTRUCTOR SCRIPT

"Every stop answers a delivery question. If time is short, keep the sprint, burndown, blockers, and risk decision; do not rush a live edit or manufacture completion to finish the story."

ASK · THEN EXPLAIN

Which route fits a sponsor with five minutes?

Sprint scope, burndown gap, At risk decision, then the reset and close.

Protect the demo. Page 21 is optional and requires explicit edit permission; all other navigation is read-only.

Exit check. Choose one route and announce its timebox before opening the first view.

Sprint board · Burndown report · Risks and blockers · Sprint effort chart

Preflight: access, tabs, and a protected baseline

Use normal browser access to both private resources. This presenter tour needs no Node, GitHub CLI, terminal, installation, or additional account; only the optional interaction needs write permission.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo

DO · CLICK PATH

1. Open the private repository and linked private organization Project 11 separately; confirm both load with the intended signed-in account.
2. Preload the repository, 03 - Sprint 1 board, and burndown report in separate tabs; choose browser zoom that keeps labels and sums readable.
3. Read the saved Sprint filter: sprint:"Sprint 1 - Citizen beta". Keep the named filter, never replace it with @current.
4. Confirm the frozen November 9 EOD snapshot: 30 sprint issues, 100 points, 40 completed, 60 remaining; check #10 is Ready and PR #52 is OPEN.
5. Hide unrelated tabs and notifications before sharing; do not display credentials. Reserve any deliberate #10 edit for page 21 only.

SHOW · FOCUS THE AUDIENCE

- Repository and Project permissions are separate checks.
- Scenario dates stay November 2–13 even when today changes.

EXPECTED · VERIFY ON SCREEN

- Both private resources load.
- No unsaved view changes.
- #10 Ready; #8 In review; PR #52 OPEN.

SAY · INSTRUCTOR SCRIPT

“This environment is already populated. We are browsing a dated delivery scenario, not setting up tools or operating a live service. If access or a baseline check fails, I will use the recovery page rather than rebuild the data.”

ASK · THEN EXPLAIN

Why not use the current-sprint shortcut?

The frozen named iteration must remain visible after its calendar dates have passed.

Protect the demo. No seeding, new issues, drafts, imports, archiving, permission changes, or credential display.

Exit check. Three readable tabs are ready; the baseline is intact before the audience arrives.

Private repository · Private Project · Named Sprint board · Burndown report

REPOSITORY ENTRY

1 min

Start at the private repository

Establish the correct starting place before showing a board. This repository owns the issues and reports; its linked organization Project supplies the planning views.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo

DO · CLICK PATH

1. On the repository Code tab, point to alvine-aurelio-org/citypulse-projects-demo and the Private badge.
2. In About, follow the Sprint board link to 03 - Sprint 1 board; confirm the address identifies organization Project 11.
3. Return to the repository and select Projects → CityPulse | Digital Service Launch; this opens the same linked Project.
4. Return to Code and scroll the README through the overview, embedded burndown, and Presentation guide; leave the report open separately.

SHOW · FOCUS THE AUDIENCE

- README: one repository, one linked Project, fifty real issues.
- About and Projects are two entry points to the same Project.
- README links lead to the report, auditable CSV, and presenter guide.

EXPECTED · VERIFY ON SCREEN

- Private repository identity matches.
- Projects opens CityPulse | Digital Service Launch, Project 11.
- README burndown and guide links are visible.

SAY · INSTRUCTOR SCRIPT

"The repository is the front door. Its issues, review artifacts, and reports are real GitHub objects. The organization-owned Project is explicitly linked here; it is not another copy of the backlog or one of the workshop lab Projects."

ASK · THEN EXPLAIN

Does opening Projects create a second backlog?

No. The linked Project presents the same repository issues through saved views.

Protect the demo. Do not navigate to another lab Project, use a template, or create a replacement repository.

Exit check. The audience can repeat the repository → Projects → CityPulse navigation unaided.

Repository Projects tab Sprint board Existing presenter guide

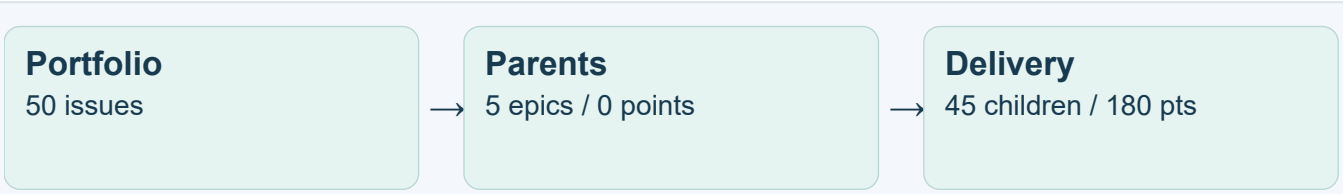
VIEW 01

1 min

Portfolio: one scope, native hierarchy

Use 01 - Portfolio backlog to connect strategic workstreams with delivery issues. Its unfiltered scope includes the five epic issues, so portfolio counts differ from the delivery board.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/1



Instructional schematic — not a GitHub screenshot. Confirm the native values in the linked live page.

DO · CLICK PATH

1. Project view tabs → 01 - Portfolio backlog; confirm the saved filter is empty.
2. Select View beside the filter; inspect Group by → Workstream and Field sum → Count plus Estimate, then dismiss unchanged.
3. Expand Citizen onboarding and any collapsed parent rows; point to Type, Parent issue, and Sub-issues progress.
4. Locate #1, Citizen onboarding, and its zero Estimate; open its title for the next stop.

SHOW · FOCUS THE AUDIENCE

- 50 issues across five Workstream groups; Count and Estimate sum answer different questions.
- Native Type, Parent issue, and Sub-issues progress are not a body checklist.
- Portfolio In progress: 10 issues, including 5 zero-point epics.

EXPECTED · VERIFY ON SCREEN

- 50 portfolio issues / 180 points.
- 5 epics / 45 native children.
- In progress: 10 including the 5 epics.

SAY · INSTRUCTOR SCRIPT

"Fifty includes the five parents. A portfolio can show an active epic and its active children without counting its effort twice: the epic estimate is zero. Native hierarchy supplies roll-up progress; the body describes the fictional scope."

ASK · THEN EXPLAIN

Why might you see fewer than fifty rows on screen?

Groups or hierarchy can be collapsed, and the table virtualizes rows; visible DOM rows are not the total.

Protect the demo. Do not count only visible DOM rows or add epic estimates to their children.

Exit check. The audience can explain why the portfolio includes parents and how zero-point roll-ups avoid double-counting.

Epic #1

Epic and child: what native completion means

Compare the Citizen onboarding parent with its first child. Separate live GitHub state, descriptive acceptance criteria, fictional roles, and the real account assigned to the issue.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo/issues/1



Instructional schematic — not a GitHub screenshot. Confirm the native values in the linked live page.

DO · CLICK PATH

1. Open #1, Citizen onboarding; inspect Type → Feature, the epic label, and the native Sub-issues section.
2. Read the native progress: 9 children, 2 closed-completed, or 2/9; locate the zero Estimate in its CityPulse Project fields.
3. Follow #6, Define the minimum-data citizen profile; compare its parent link, Done Project status, closed-completed issue state, and 5-point Estimate.
4. Read Acceptance criteria and Fictional role and real assignee in the body; compare Profile policy designer with the actual Assignees account alvinea28.

SHOW · FOCUS THE AUDIENCE

- Feature is the native type; epic is a label, not an organization Epic type.
- #6 describes mandatory routing, optional profile data, and a correction or deletion path.
- Checked body criteria are seeded descriptions; native issue state is independently visible.

EXPECTED · VERIFY ON SCREEN

- #1: Feature + epic label; 9 children; 2/9 complete.
- #6: Done; closed-completed; 5 points.
- Actual assignee: alvinea28 only.

SAY · INSTRUCTOR SCRIPT

"The child really is closed in GitHub, but that does not prove a profile service was built. Its acceptance criteria are scenario content. The native parent-child links and state are real; the policy designer is a fictional role, not another account."

ASK · THEN EXPLAIN

Should we add five points to the epic as well?

No. Count the child once; the epic stays at zero points.

Protect the demo. Do not tick criteria, edit bodies, or treat a seeded checkbox as test or approval evidence.

Exit check. Distinguish native hierarchy and state from the fictional narrative before moving to the delivery board.

Child #6

VIEW 02

2 min

Delivery board: six stages, 45 children

Show the full child-work lifecycle before narrowing to the sprint. The saved -label:epic filter removes the five parents; all 45 delivery children remain.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/2

DO · CLICK PATH

1. Project view tabs → 02 - Delivery board; read -label:epic in the filter bar.
2. Scan the six Status columns from Backlog through Done; compare each Count and Estimate sum with the table.
3. Open #10, Correct screen-reader association of profile errors, in Ready; show its real issue link, Bug type, parent, and 2-point Estimate.
4. Close the detail panel without editing; compare the five In progress children with the portfolio total of ten.

SHOW · FOCUS THE AUDIENCE

- Equal card counts can hide different effort.
- Backlog has 65 points; Ready has 25 despite both having 10 issues.
- Card details refer to the original repository issue.

EXPECTED · VERIFY ON SCREEN

- 45 delivery children / 180 points.
- 6 Status columns; no epic cards.

Status	Issues	Points
Backlog	10	65
Ready	10	25
In progress	5	25
In review	5	15
Blocked	5	10
Done	10	40

SAY · INSTRUCTOR SCRIPT

"This is the delivery lifecycle, not the sprint commitment. Backlog, Ready, active work, review, blockers, and Done remain distinct. Moving a card changes planning state; it does not demonstrate that fictional functionality works."

ASK · THEN EXPLAIN

Why are only five issues In progress here?

The other five were epic parents excluded by this view.

Protect the demo. Do not drag cards or move Done work; native automation can alter the underlying issue state.

Exit check. Explain the difference between issue counts and point sums before selecting the sprint view.

Inspect #10 without editing

VIEW 03

3 min

Sprint 1: a 100-point commitment

Narrow the delivery conversation to the one saved iteration. This scope, not the whole backlog, supplies the 100-point burndown baseline.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/3

DO · CLICK PATH

1. Project view tabs → 03 - Sprint 1 board; read sprint:"Sprint 1 - Citizen beta" in the saved filter.
2. Inspect the Sprint field on #10 and the iteration dates: November 2–13, 2026, 12 calendar days inclusive and 10 working days.
3. Close the issue panel; compare the board Count and Estimate sums with the five occupied columns below; Backlog has no sprint items.
4. Point to #8 in In review and #11 in Blocked; neither receives completed credit. Keep all statuses unchanged.

SHOW · FOCUS THE AUDIENCE

- 30 committed issues / 100 points.
- 10 Done issues / 40 points; 20 open issues / 60 points remaining.
- 5 epics and 15 uncommitted children are excluded.

EXPECTED · VERIFY ON SCREEN

- 30 sprint issues / 100 points.
- 10 Done / 40 completed points.
- 20 open / 60 remaining points.

Sprint status	Issues	Points
Ready	5	10
In progress	5	25
In review	5	15
Blocked	5	10
Done	10	40

SAY · INSTRUCTOR SCRIPT

“The commitment is one hundred points, not one hundred percent delivered. Forty points are Done; the other sixty remain in full. Reviews and active work earn no partial credit, and adding the portfolio or follow-on backlog would change the question.”

ASK · THEN EXPLAIN

Can the 15 review points reduce remaining work today?

No. They stay in remaining work until Done.

Protect the demo. Keep the explicit named Sprint filter; never substitute @current or pull backlog into this frozen commitment.

Exit check. State 100 committed, 40 completed, 60 remaining before opening the burndown.

Sprint effort chart

REPOSITORY REPORT

2 min

Burndown: 60 remaining, 20 behind ideal

Use the actual repository SVG for the conventional remaining-points story. This is illustrative history, not a native historical burndown.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo/blob/main/reports/sprint-1.md



Actual repository chart artifact. Earlier completion dates are illustrative; future actuals are blank. Open the live report for full-size inspection.

DO · CLICK PATH

1. Repository README → Sprint report and daily numbers; keep the rendered burndown visible.
2. Trace 100 kickoff points to November 9 EOD, working day 6: actual 60 remaining versus ideal 40.
3. Point to the stopped actual line and blank future actuals; open Auditable CSV to inspect the same rows and provenance.

SHOW · FOCUS THE AUDIENCE

- 40 completed points = 40% of scope.
- 60 actual minus 40 ideal = 20 points behind.

EXPECTED · VERIFY ON SCREEN

- Kickoff 100; November 9 EOD actual 60; ideal 40.
- Future actuals blank; no partial completion credit.

SAY · INSTRUCTOR SCRIPT

"These earlier completion dates are simulated. GitHub events were not backdated. The actual line stops at the scenario cutoff; no future finish is invented. Native Insights will show the real event boundary separately."

ASK · THEN EXPLAIN

Is the future blank a zero?

No. It means not observed, not completed or forecast.

Protect the demo. Illustrative dates are not native GitHub closure history or proof of product delivery.

Exit check. Explain the 20-point gap without claiming native multi-day delivery.

Auditable CSV

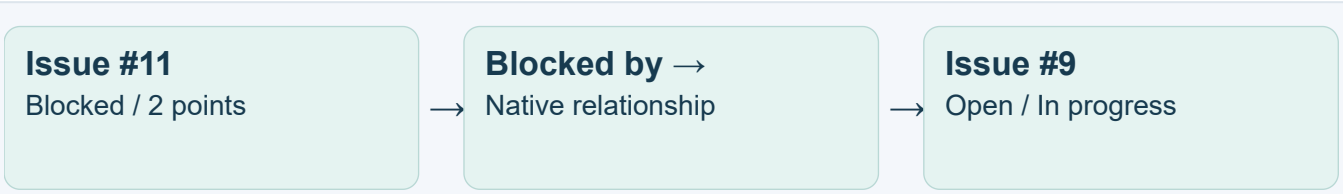
VIEW 06

2 min

Blockers: follow the unresolved prerequisite

Use 06 - Risks and blockers to turn a red status into a concrete dependency conversation. Its five blocked issues are a subset of unfinished sprint work.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/6



Instructional schematic — not a GitHub screenshot. Confirm the native values in the linked live page.

DO · CLICK PATH

1. Project view tabs → 06 - Risks and blockers; read status:Blocked and the Workstream columns, not Status columns.
2. Open #11, Resolve preference overwrite after a slow save; locate the native Blocked by relationship to #9.
3. Follow #9, Draft the notification preference editing contract; confirm it is open and In progress, and inspect its reciprocal Blocks link to #11.
4. Open #8, Review the guided profile setup journey, and follow its existing prerequisite #7, Specify consent and profile retention choices; #7 is Done and closed-completed.

SHOW · FOCUS THE AUDIENCE

- 5 blocked issues / 10 points, inside the 60 remaining points.
- #11 waits on unresolved #9; #8 has a resolved prerequisite #7.
- 15 native dependency relationships exist across the whole portfolio.

EXPECTED · VERIFY ON SCREEN

- 5 blocked issues / 10 points.
- #11 Blocked by #9; #9 open / In progress.
- #8 depends on #7; #7 Done / closed-completed.

SAY · INSTRUCTOR SCRIPT

"Blocked is not just a red label. This issue has an unresolved native prerequisite, and the prerequisite links back. Compare the resolved consent dependency: a relationship can remain visible after its prerequisite is Done without creating a new blocker."

ASK · THEN EXPLAIN

Should we report seventy points remaining?

No. The ten blocked points are already included in the sixty remaining.

Protect the demo. Inspect existing relationships only; do not add, remove, or resolve links to make the story simpler.

Exit check. Name the unresolved prerequisite before returning to the Project risk update.

Blocked #11 Prerequisite #9 Review #8 Resolved prerequisite #7

At risk: turn evidence into the next action

Show the existing native Project update, not a newly composed presentation note. Connect the burndown gap and unresolved prerequisites to a practical planning decision.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11

DO · CLICK PATH

1. Return to Project 11; select the At risk status in the Project toolbar, then open the latest existing status update.
2. Read Sprint 1: citizen beta is at risk and its November 13 target; locate the dated fictional November 9 EOD disclosure.
3. Point to Risk, Mitigation, and Next action: inspect blockers, finish current prerequisites and review work, then reconfirm beta scope.
4. Follow its Single-sprint burndown and provenance link if challenged about the dates; return without posting an update.

SHOW · FOCUS THE AUDIENCE

- 60 remaining versus 40 ideal explains the 20-point gap.
- Five unresolved prerequisites block five issues / 10 points.
- The next action is a scope and dependency decision, not an invented recovery promise.

EXPECTED · VERIFY ON SCREEN

- Native status: At risk.
- Target: November 13, 2026.
- Risk, Mitigation, and Next action are present.

SAY · INSTRUCTOR SCRIPT

"At risk is a useful conversation when it states the evidence, mitigation, and next decision. Here we should finish integration prerequisites and review work before adding backlog. The November thirteenth target remains a planning gate, not a promise that a service will launch."

ASK · THEN EXPLAIN

What should the next stand-up focus on?

The unresolved prerequisites, finishing review work, and reconfirming beta scope.

Protect the demo. The update describes a fictional scenario; do not create or edit an update during presentation.

Exit check. The audience can repeat the risk and next action without treating the date as delivery evidence.

Burndown and provenance

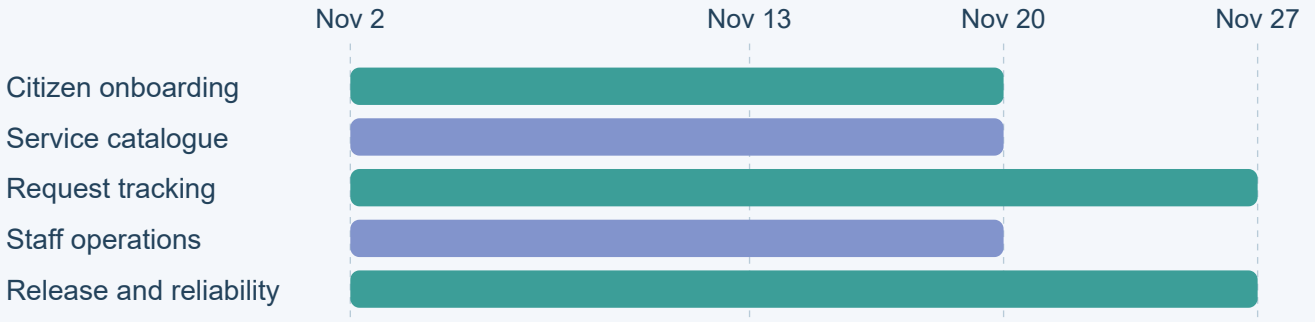
VIEW 04

2 min

Roadmap: planned dates, not delivery evidence

Show five epic bars as a release-horizon conversation. The saved roadmap filters to epics and maps explicit planning fields rather than native creation timestamps.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/4



Instructional schematic — not a GitHub screenshot. Confirm the native values in the linked live page.

DO · CLICK PATH

1. Project view tabs → 04 - Release roadmap; confirm label:epic and locate all five epic bars, including Citizen onboarding.
2. View beside the filter → Dates; inspect Start date and Target date, then dismiss without choosing alternatives.
3. Inspect Markers → Sprint and Milestone; leave selections unchanged and do not use Save view.
4. Point to Operations readiness on November 20 and Public launch on November 27; use the repository Milestones page next for the Citizen beta gate.

SHOW · FOCUS THE AUDIENCE

- 5 epic bars; planning starts November 2 and targets extend to November 20 or 27.
- Markers depend on the filtered items, not just milestone existence.
- Citizen beta, November 13, may not appear here: no epic is assigned to it.

EXPECTED · VERIFY ON SCREEN

- 5 epic bars.
- Dates: Start date + Target date.
- Markers: Sprint + Milestone.

SAY · INSTRUCTOR SCRIPT

"The bars are planned dates, not createdAt timestamps or actual delivery. This epic-only view can show later milestone markers while omitting Citizen beta. The gate still exists; its sprint children, rather than these epics, belong to it."

ASK · THEN EXPLAIN

Does a November twenty-seventh bar prove public launch?

No. It records a planning horizon, not completion, approval, or deployment.

Protect the demo. Inspect controls read-only; do not move bars, change dates, or save exploratory view changes.

Exit check. Explain the missing beta marker before opening the repository milestone list.

Repository milestones

REPOSITORY PLANNING

1 min

Milestones: three gates, not three sprints

Separate repository release gates from the Project iteration. Citizen beta happens to cover the same 30 issues as Sprint 1; that does not make the two concepts interchangeable.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo/milestones

DO · CLICK PATH

1. Repository Issues → Milestones, or open the linked milestone list; locate Citizen beta, Operations readiness, and Public launch.
2. Open Citizen beta and compare its 10 closed / 20 open issues with the Sprint board commitment.
3. Return to the milestone list; read the November 13, 20, and 27 due dates and the later-gate counts below.
4. Point to beta completion as 10 of 30 issues; compare this count ratio with 40 of 100 sprint points without editing the milestone.

SHOW · FOCUS THE AUDIENCE

- Issue-count progress is not point-weighted progress.
- Open dependencies still matter even when a due date is near.
- Later gates include follow-on children and epic issues.

EXPECTED · VERIFY ON SCREEN

- 3 milestone gates, separate from 1 sprint.
- Citizen beta: 30 issues, 10 closed / 20 open.

Milestone	Due in 2026	Issues	Closed	Open
Citizen beta	November 13	30	10	20
Operations readiness	November 20	16	0	16
Public launch	November 27	4	0	4

SAY · INSTRUCTOR SCRIPT

"A sprint timeboxes a commitment; a milestone groups issues around a decision gate. Ten of thirty beta issues are closed, while forty percent of its estimated points are Done. Neither percentage removes the unresolved prerequisites."

ASK · THEN EXPLAIN

Why can beta show about one-third closed but forty percent Done?

The milestone counts issues; the sprint effort calculation weights their estimates.

Protect the demo. Do not edit due dates or close milestones; these are illustrative planning gates, not release approvals.

Exit check. Distinguish issue-count progress, point completion, and the release decision.

Compare Sprint scope

VIEW 05

2 min

Workload: distribution is not capacity

Use open delivery work to discuss distribution across the five fictional teams. This view includes follow-on backlog, so its scope is wider than unfinished Sprint 1.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/5

DO · CLICK PATH

1. Project view tabs → 05 - Team workload; read is:open -label:epic in the saved filter.
2. Select View beside the filter; inspect Group by → Team and Field sum → Count plus Estimate, then dismiss.
3. Read each group total against the table; compare 35 open children / 140 points with the sprint-only 20 open / 60 points.
4. Open Ready #10 to show no assignee, then In progress #9 to show alvinea28; return without assigning either item.

SHOW · FOCUS THE AUDIENCE

- Experience carries 52 points; Product carries 26.
- Team values and delivery roles are fictional, not GitHub access-control groups.
- Unassigned Ready and Backlog work is intentional.

EXPECTED · VERIFY ON SCREEN

- 35 open delivery issues / 140 points.
- Sprint subset: 20 open issues / 60 points.
- Only actual assignee: alvinea28.

Fictional Team	Open issues	Points
Product	5	26
Experience	10	52
Platform	8	28
Service Ops	7	20
Quality	5	14

SAY · INSTRUCTOR SCRIPT

"These totals reveal distribution, not available capacity or measured velocity. The same real operator appears on assigned examples; we have not invented team accounts. Before promising a date, we would need actual staffing, availability, and delivery evidence."

ASK · THEN EXPLAIN

Can Experience safely accept fifty-two points next sprint?

This view cannot tell us; it shows open workload, not capacity or throughput.

Protect the demo. Do not assign fictional people, change Team, or present estimates as proven capacity.

Exit check. State which open-work scope the 140 points represents before comparing it with Insights.

Team workload chart

VIEW 07

2 min

Bug triage: discuss the synthetic privacy case

Show how native issue type and planning priority answer different questions. The privacy example is synthetic content designed for discussion, not a production incident or an observed leak.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/7

DO · CLICK PATH

1. Project view tabs → 07 - Bug triage; read type:Bug and inspect Priority grouping plus the native Type column.
2. Open #28, Remove internal staff notes from citizen message examples; point to Bug, P0 - Critical, Ready, and 2 points.
3. Read its existing acceptance criteria: synthetic internal text is separate, only citizen-visible fields are permitted, and missing public text gets a neutral fallback.
4. Ask the audience to describe the unsafe missing-message case and safe fallback; dismiss the detail panel without ticking criteria or changing priority.

SHOW · FOCUS THE AUDIENCE

- 8 native Bug issues across delivery stages.
- P0 expresses fictional privacy impact; it is not a native issue type.
- Feature and Task work sits outside this Bug-only filter.

EXPECTED · VERIFY ON SCREEN

- 8 native Bugs.
- #28: Ready; P0 - Critical; 2 points; unassigned.
- Missing public text has a neutral fallback criterion.

SAY · INSTRUCTOR SCRIPT

"If public wording is missing, copying an internal note would violate the proposed boundary. A neutral fallback is the recoverable case. We are discussing acceptance conditions for a fictional defect, not claiming a tested fix, an actual disclosure, or an implemented notification service."

ASK · THEN EXPLAIN

What should happen when the public message is absent?

Use a neutral fallback; never insert internal staff-note content.

Protect the demo. Do not describe this synthetic case as a real incident, completed fix, or executed test.

Exit check. The audience can distinguish issue type, priority, and a proposed acceptance condition.

Synthetic privacy Bug #28

2 min

Insights: choose Count or Sum deliberately

Compare the three saved current-state charts. Read the aggregation and scope before interpreting the bars; the same Status category can describe counts or estimated points.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/insights/2

DO · CLICK PATH

1. Project toolbar → Insights → Sprint 1 - Effort by status; open Configure only to read Layout, X axis, Y axis, and the Estimate field.
2. Confirm Column, X Status, Y Sum of Estimate, and sprint:"Sprint 1 - Citizen beta"; compare Ready 10, In progress 25, In review 15, Blocked 10, Done 40.
3. Select Delivery status; inspect Column, X Status, Y Count, no filter: its bars total 50 issues, not 100 points.
4. Select Team workload; inspect Column, X Team, Y Sum of Estimate, is:open -label:epic; compare Product 26, Experience 52, Platform 28, Service Ops 20, Quality 14.

SHOW · FOCUS THE AUDIENCE

- Read scope first, then axis, then aggregate field.
- Counts include zero-point epics; delivery estimates do not gain extra epic effort.

EXPECTED · VERIFY ON SCREEN

- Delivery status: 50 issues.
- Sprint effort: 100 points.
- Team workload: 140 points.

Saved chart	Measure	Scope total
Delivery status	Count / Status	50 issues
Sprint 1 - Effort by status	Sum of Estimate / Status	100 points
Team workload	Sum of Estimate / Team	140 points

SAY · INSTRUCTOR SCRIPT

"A chart can be correct while answering the wrong question. Count answers how many issues; Sum of Estimate answers how much estimated work. The saved filters distinguish the whole portfolio, the committed sprint, and all open delivery work."

ASK · THEN EXPLAIN

Which chart answers sprint effort remaining?

Sprint 1 - Effort by status: 100 total, 40 Done, 60 not Done.

Protect the demo. Configure is for inspection only; do not change selections or save exploratory chart changes.

Exit check. Name the aggregate and filter before quoting any chart value.

Delivery status · Sprint 1 - Effort by status · Team workload

CUSTOM CHART 04

1 min

Native burn-up: read the real event boundary

Use the saved native history chart to explain why an illustrative burndown and real GitHub event history need different provenance. They agree on the snapshot, not on earlier delivery dates.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/insights/4

DO · CLICK PATH

1. Project toolbar → Insights → Sprint 1 - Native burn up, the custom chart at insights/4; do not confuse it with the retained default Burn up.
2. Inspect Configure read-only: Stacked area, X Time, Y Sum of Estimate, and the named Sprint 1 filter.
3. Read the saved November 2–9 range; hover or focus November 9 to show Open 60 and Completed 40.
4. Point to zero earlier values before native creation on November 9, then compare the repository burndown disclosure without altering chart dates.

SHOW · FOCUS THE AUDIENCE

- Native creation and closure events occurred November 9, 2026.
- Earlier zero values are not proof that the sprint had no work.
- The repository report uses declared simulated daily completion dates.

EXPECTED · VERIFY ON SCREEN

- Stacked area / Time / Sum of Estimate.
- Named Sprint filter; November 2–9 range.
- November 9 only: Open 60 / Completed 40; earlier values zero.

SAY · INSTRUCTOR SCRIPT

"This is native burn-up, not native burndown. The environment was created on November ninth, so GitHub cannot supply real earlier delivery for it. We have not backdated events or manufactured history; the separate burndown explicitly labels its illustrative dates."

ASK · THEN EXPLAIN

Why does this chart not trace the earlier report completions?

Native history uses actual GitHub events, not the catalogue simulated completion dates.

Protect the demo. Do not call this multi-day delivery evidence, a native burndown, or a backdated event record.

Exit check. The audience can explain both charts without mistaking their different histories.

Illustrative burndown comparison

CODE-TO-WORK TRACE

2 min

Pull requests: merged assembly, open design

Contrast the real merged demo assembly with an intentionally open design proposal. Neither PR is an extra Project card, and the proposed design is not an implemented service.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo/pull/52



Instructional schematic — not a GitHub screenshot. Confirm the native values in the linked live page.

DO · CLICK PATH

1. Repository Pull requests → Closed → PR #51; read Merged and inspect the real demo-assembly diff and checks.
2. Open PR #52; read OPEN in Conversation and the Closes #8 reference; leave it open.
3. Select Files changed to inspect the proposed profile-setup design document, then Checks for this PR head; return to Conversation.
4. Follow #8, Review the guided profile setup journey; show the native Development link back to PR #52 and its In review Project status.

SHOW · FOCUS THE AUDIENCE

- PR #51 merged the demo assembly, not a citizen-service runtime.
- PR #52 proposes a design and remains OPEN.
- Green CI is not human approval; no runtime service is present.

EXPECTED · VERIFY ON SCREEN

- PR #51 MERGED; PR #52 OPEN.
- PR #52 has Closes #8; #8 remains open / In review.
- Both PRs outside the 50 issue cards.

SAY · INSTRUCTOR SCRIPT

"The assembly merge is real repository work. The profile proposal demonstrates traceability from a design diff to an issue awaiting review. Its closing reference would affect issue eight if merged, so we keep that decision deliberately open and preserve the sprint snapshot."

ASK · THEN EXPLAIN

Why not complete the story by merging the proposal?

It is intentionally open; merging could close #8 and change the frozen commitment metrics.

Protect the demo. Never merge PR #52, close #8, or add either PR to the Project for this presentation.

Exit check. Leave PR #52 OPEN and #8 In review; explain checks versus approval.

Merged assembly PR #51 Issue #8 in review

REPOSITORY VALIDATION

1 min

Actions: green checks at the right revision

Show what the hosted checks actually validate and bind each result to the revision being discussed. A green run on another branch or an old commit does not validate the current one.

Open live page → github.com/alvine-aurelio-org/citypulse-projects-demo/actions

DO · CLICK PATH

1. Repository Code tab → latest main commit; note its revision, then Actions → Demo quality → successful main run at that same revision.
2. Open the job and expand its test and report-freshness steps; read the actual pass/fail results shown, not a fixed count from this guide.
3. In Actions, open the successful CodeQL checks for the same current main revision; confirm the branch and commit before quoting the result.
4. Return to PR #52 → Checks for proposal-specific results; distinguish that PR head from main. If a matching run is not green, state the gap rather than substitute an older success.

SHOW · FOCUS THE AUDIENCE

- Demo quality checks catalogue integrity, maths, report freshness, and provisioning safety.
- CodeQL results belong to a specific source revision.
- Logs validate this repository tooling, not fictional resident journeys.

EXPECTED · VERIFY ON SCREEN

- Demo quality and CodeQL successful at the same current main revision.
- PR checks, when discussed, match the PR head.

SAY · INSTRUCTOR SCRIPT

"Green checks are useful when we identify the exact revision and what ran. These checks cover the demo and its tooling; they are not runtime application tests, a security guarantee, a human design approval, or production release permission."

ASK · THEN EXPLAIN

Does a green main run approve the open design PR?

No. Check that PR head separately; approval remains a human decision.

Protect the demo. Read existing logs only; do not rerun workflows or claim application health from repository CI.

Exit check. Name the revision, checks, and evidence boundary without hardcoding the current test count.

Open proposal checks

Workflows: inspect the saved rules safely

Inspect the seven enabled native rules before explaining automation. Project workflows act on real issues, so a presentation-only status move can have consequences beyond the board.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/workflows

DO · CLICK PATH

1. Project toolbar → Workflows (7 enabled), or open the canonical workflows link.
2. Select Item closed and read its Done action; return to the list and inspect Item reopened → Ready.
3. Inspect Pull request linked → In review and Auto-close issue; explain that choosing Done can close the underlying issue.
4. Read the other enabled rule names below; verify broad Auto-add to project and Auto-archive items are off, then leave all switches unchanged.

SHOW · FOCUS THE AUDIENCE

- Seven enabled rules, including sub-issue auto-add.
- Broad Auto-add and Auto-archive are off; neither is needed for this fixed scope.
- An automatic native state change is not evidence that fictional work was implemented.

EXPECTED · VERIFY ON SCREEN

- 7 enabled native workflows.
- Broad Auto-add off; Auto-archive off.

Enabled rule	Inspect only
Item closed	Status becomes Done
Auto-close issue	Done can close the issue
Item reopened	Status becomes Ready
Pull request linked	Linked issue becomes In review
Pull request merged	Read its saved trigger and action
Item added to project	Read its saved trigger and action
Auto-add sub-issues to project	Read its saved trigger and action

SAY · INSTRUCTOR SCRIPT

"We inspect the current saved rules instead of assuming automation is harmless. Closing, reopening, or linking a PR can change status, and Done can close an issue. That is why the optional example only moves Ready to In progress and back."

ASK · THEN EXPLAIN

Why not use Done for a dramatic card move?

The enabled auto-close rule can close the real issue and change the snapshot.

Protect the demo. Do not enable, disable, edit, archive, or trigger rules to decorate the demo.

Exit check. Leave every saved rule unchanged; use page 21 only if explicit permission allows its reversible move.

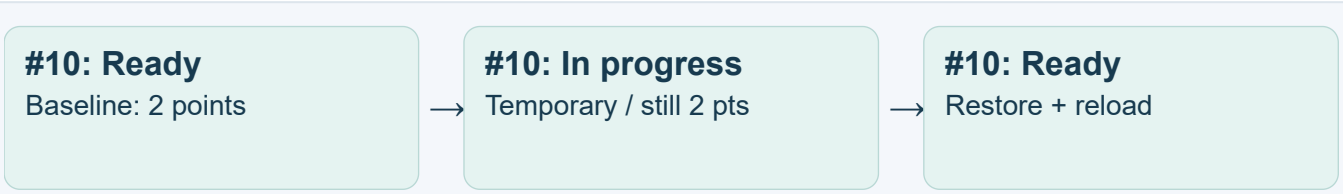
EXPLICIT OPT-IN ONLY

Optional 2 min

Optional: move #10, verify, then restore

Only with explicit owner permission. Demonstrate persistence with one known open issue, then restore it before discussing frozen metrics.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/3



Instructional schematic — not a GitHub screenshot. Confirm the native values in the linked live page.

DO · CLICK PATH

1. In 03 - Sprint 1 board open #10, Correct screen-reader association of profile errors; verify Ready, 2 points, unassigned, and Sprint 1 - Citizen beta.
2. CityPulse Project fields → Status dropdown → In progress; close the panel, reopen the canonical Sprint board link, and reload.
3. Compare temporary column sums with the checkpoints below; Done and remaining points must not change.
4. Reopen #10 → CityPulse Status dropdown → Ready; close the panel, reopen the canonical Sprint board link, and reload again.
5. Verify restored sums and #10 fields below; stop if anything else changed.

SHOW · FOCUS THE AUDIENCE

- Persistence needs canonical navigation and reload, not an animation.
- Changing active stage earns no completed credit.

EXPECTED · VERIFY ON SCREEN

- Temporary Ready: 4 issues / 8 points; In progress: 6 / 27.
- Restored Ready: 5 issues / 10 points; In progress: 5 / 25.
- Done 40 points / remaining 60 throughout.
- #10 restored: open, Ready, 2 points, unassigned, same named Sprint.

SAY · INSTRUCTOR SCRIPT

"Only this two-point Ready issue is moving. The sprint total and completed work stay unchanged. I will now restore Ready and verify the persisted state so the next instructor sees the same baseline."

ASK · THEN EXPLAIN

Should remaining points fall when work starts?

No. They stay sixty until work is Done; this move earns no partial credit.

Protect the demo. Never choose Done or edit Assignees, Sprint, or Estimate; automation can close issues.

Exit check. After reload, #10 is Ready and every baseline field and sprint sum is restored.

Canonical Sprint board for both reloads Only permitted issue #10

Audience questions: answer with the right scope

Use these answers during discussion instead of changing filters or inventing evidence. Keep the named Sprint board visible as the common reference.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/3

DO · CLICK PATH

1. Return to 03 - Sprint 1 board; leave its named filter and sums unchanged.
2. Ask which scope or measure the question concerns before quoting a number.
3. Use the table, then return to the relevant printed stop if the audience wants the native evidence; do not demonstrate by mutating data.

SHOW · FOCUS THE AUDIENCE

- Scope and measure must travel with every number.
- A planning signal is not proof of capacity, approval, or deployment.

EXPECTED · VERIFY ON SCREEN

- Counts and points remain distinct.
- Blocked 10 is a subset of remaining 60.

Audience question	Instructor answer
Why 50, 45, 30, and 35?	50 includes 5 epics; 45 delivery children; 30 committed sprint children; 35 open delivery children across all horizons.
Why 100, 180, and 140 points?	100 committed sprint points; 180 all delivery points; 140 open delivery points. Zero-point epics add no effort.
How much is complete?	40 Done points / 100 = 40%; 60 remain. Active and review stages earn no partial credit.
Should blockers be added to remaining?	No. The 10 blocked points are inside the remaining 60.
Is Team workload capacity or velocity?	Neither. Fictional team groupings show open estimated work, not actual staffing or measured throughput.
Does green CI approve or deploy the service?	No. Revision-specific repository checks are not human approval, runtime acceptance, or production authorization.
Is the PDF burndown native historical delivery?	No. It uses illustrative dates; native events start November 9. Native Insights is burn-up, not this burndown.

SAY · INSTRUCTOR SCRIPT

"The apparently conflicting figures answer different questions. I will name the population and unit first, then explain the evidence boundary. We do not need another issue, a manufactured completion, or an unsaved filter to make the answer convincing."

ASK · THEN EXPLAIN

Which effort scope belongs in the sprint stand-up?

100 committed points: 40 Done and 60 remaining, not all 180 delivery points or 140 open-work points.

Protect the demo. Do not convert a fictional scenario answer into a claim of tested or deployed functionality.

Exit check. Answer one scope question and one provenance question before the final reset.

Recover without reseeding or guessing

Pause on unexpected state. Fix navigation or discard your own unsaved presentation changes; do not overwrite unfamiliar edits or rebuild the environment.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11

DO · CLICK PATH

1. 404: verify the intended account has separate read access to both private resources; do not assume deletion.
2. Empty Sprint: reopen 03 - Sprint 1 board with sprint:"Sprint 1 - Citizen beta", not @current. Missing rows: expand Workstream groups and parents; virtualized rows are not missing issues.
3. Wrong chart: inspect Count versus Sum of Estimate and filter. Wrong roadmap: inspect View beside the filter → Dates → Start date + Target date; dismiss unchanged.
4. Unsaved filters/display: discard, do not Save. Changed baseline: restore only your own page 21 #10 move; otherwise stop and ask the owner, never seed.
5. Optional export: count loaded view rows before the selected view options → Export view data → Export confirmation; verify a completed saved download, never confirmation alone.

SHOW · FOCUS THE AUDIENCE

- Use canonical links, not another Project or a replacement issue.
- Keep the dated PDF as a fallback; distinguish it from the live browser.

EXPECTED · VERIFY ON SCREEN

- Correct private account access checked.
- Named filter and saved axes inspected.
- No blind reset, seeding, or assumed export success.

SAY · INSTRUCTOR SCRIPT

"Pause rather than erase someone else's work. If access fails, I can explain the dated PDF without claiming I verified a live screen or completed download."

ASK · THEN EXPLAIN

Does clicking Export prove the TSV was saved?

No. Check completed browser download evidence and the saved file, or report it unconfirmed.

Protect the demo. Never recreate, import, archive, or change permissions as presentation recovery.

Exit check. Resume only with a known state; otherwise explain the limitation and leave unfamiliar changes intact.

Canonical Sprint board

LEAVE IT REPEATABLE

1 min

Quick reference: reset and close on the risk

Close on the named Sprint board and a decision, not a fabricated happy ending. Use the compact links to revisit the same native objects.

Open live page → github.com/orgs/alvine-aurelio-org/projects/11/views/3

50	45	30	35
portfolio issues	delivery children	sprint issues	open delivery

DO · CLICK PATH

1. Open 03 - Sprint 1 board and reload its canonical link.
2. Check no unsaved changes, #10 Ready, #8 In review, and PR #52 OPEN; restore only your own optional edit if needed.
3. Recap At risk: resolve prerequisites, finish reviews, and reconfirm beta scope; leave this board on screen.

SHOW · FOCUS THE AUDIENCE

- Scopes: 50 portfolio / 45 delivery / 30 sprint / 35 open delivery.
- Delivery counts: Backlog 10; Ready 10; In progress 5; In review 5; Blocked 5; Done 10.

EXPECTED · VERIFY ON SCREEN

- Sprint: 100 committed / 40 completed / 60 remaining.
- No unsaved changes; #10 Ready; #8 In review; PR #52 OPEN.

SAY · INSTRUCTOR SCRIPT

"One connected environment makes the scope, commitment, blockers, and next decision visible. It does not prove a service was built or deployed. We leave the risk explicit and the baseline intact for the next discussion."

ASK · THEN EXPLAIN

What is the final delivery message?

100 committed, 40 completed, 60 remaining; focus on prerequisites and review work, not more backlog.

Protect the demo. Do not merge, close, archive, or claim deployment to make the ending look better.

Exit check. Leave Sprint board 03 reloaded, unchanged, and visibly At risk.

V1 Portfolio
V2 Delivery
V3 Sprint
V4 Roadmap
V5 Workload
V6 Blockers
V7 Bugs
C1 Status count
C2 Sprint effort
C3 Team effort
C4 Native burn-up
#8 In review
#10 Reset
#11 Blocker
PR #51 Merged
PR #52 OPEN
Burndown report